

Advantages of MongoDB

Sure! Let's dive into the concept of **flexible schema in MongoDB**.

In simple terms, a flexible schema means that you can store data in a way that doesn't require a strict structure. Imagine you have a box where you can put different types of toys—some are big, some are small, and some are even different shapes. You don't need to worry about fitting them into specific compartments; you can just toss them in as they are. This is similar to how MongoDB allows you to store data. For example, if you have addresses from different countries, some might not have a zip code, while others might not have a postal code. In a traditional database, this would create problems, but in MongoDB, you can simply store the data without needing to create extra fields for missing information.

This flexibility is especially useful when dealing with unstructured data, like combining information from various sources. It allows you to adapt quickly to changes without having to redesign your entire database. So, if you suddenly need to add new information, like contactless delivery details, you can do that easily without any hassle!

Flexibility with schema

```
{  
  "street": "10 High St",  
  "city": "London",  
  "postcode": "W1 1SU"  
}
```

```
{  
  "street": "8717 West St",  
  "city": "New York",  
  "zip": "10940"  
}
```

MongoDB allows flexibility with the schema

- The first key benefit of using Mongo DB is the flexibility of its schema. Compare the two addresses shown here. There is no zip code for UK addresses and no post code for American addresses. In a relational world where fields names must be present in each row, this would cause us to have either overarching fields or a lot of fields with no values.
- But storing in this format in MongoDB is not a problem because it allows us this flexibility with the schema. This also allows us to store unstructured data, for example combining data of different shapes from different sources for analysis or storage.

Code-first approach

In relational databases

- Design
- Then code

```
CREATE TABLE Students (  
    FirstName varchar(255),  
    LastName varchar(255),  
    Email varchar(255),  
    StudentId int);
```

```
INSERT INTO Students  
VALUES (  
    "John",  
    "Doe",  
    "john.doe@email.com",  
    20217484);
```

- The second benefit of using MongoDB is its code-first approach. Relational databases add complexity when you start because first you need to create your table design and only then are you allowed to work on your database. Since Mongo DB works on documents, it means you can access your data without having any complexity in between.

Code-first approach

In relational databases

- Design
- Then code

In MongoDB, code first!

- No complex table definitions
- Write as soon as you connect to DB

```
db.persons.insertOne({
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@email.com",
  "studentId": 20217484
})
```

- There are no complex table definitions. You can start writing your first data as soon as you connect to your MongoDB database. This also eliminates the use of any third-party frameworks to give you read and write operations.

Evolving schema

The whole world changed in 2020!

```
//pre-covid schema
{
  "street": "10 High St",
  "city": "London",
  "postcode": "W1 1SU"
}
```

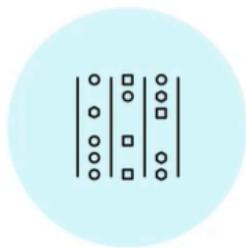
```
//evolved schema
{
  "street": "10 High St",
  "city": "London",
  "postcode": "W1 1SU",
  "contactlessDelivery": true
}
```

- The third key benefit is an evolving schema. Imagine you run a courier company, so you need to store delivery addresses. With the massive changes in delivery procedures introduced in 2020, you need to quickly evolve your schema to store additional information about contactless delivery with minimal disruption. With MongoDB this is easily possible. All you need is to start storing this additional information in your delivery documents.
- Let's look at an example of a stock market aggregator. This collects stock data from different sources. Each piece of source data can be a different shape, which means it's essentially unstructured data when it comes to MongoDB. With the flexibility MongoDB provides, we can store all this data together in one collection.

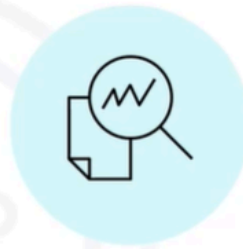
Querying and analytics

MongoDB querying using MQL

Has a wide range of operators



For complex analysis use aggregation pipelines

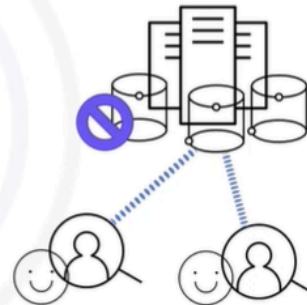


- The next key benefit is querying and analytics capabilities. Mongo Query Language or MQL has a number of operators that you can use to build fairly complex queries to find your data. If your queries are even more complex, you can use aggregation pipelines within MongoDB. For example, group all student results by their year group and find top scoring students by semester.

High availability

MongoDB is natively a highly available system:

- Resilience through redundancy
- No system maintenance downtime
- No upgrade downtime



- And the last key benefit we will discuss is high availability. MongoDB natively supports being a highly available system by means of redundancy. Typical MongoDB setups are three-node replica sets where one of them is a primary member and the others are secondary members. Replication keeps a copy of your data on other data-bearing nodes in the cluster. If one system fails, the other one takes over and you do not see any downtime.
- This also works for system maintenance situations where you might take away a node from making updates to software, operating systems, file systems, security patches, or your MongoDB version.

Recap

In this video, you learned that:

- The database schema can be flexible when working with MongoDB
- MongoDB uses a code-first approach
- MongoDB also utilizes an evolving schema
- Complex data analysis can be done on the server using aggregation pipelines
- MongoDB provides native high-availability